UNIT IV: Recurrent Neural Network (RNN)

---

1. Define sequence learning and give two examples of sequence-based tasks.

Sequence learning is a type of machine learning where the model works with data that is sequential in nature. In such data, the order of the elements is crucial for understanding its meaning and context. The goal is to learn a mapping from an input sequence to an output, which can be a single value or another sequence. Unlike traditional models that assume inputs are independent of each other, sequence learning models are designed to capture temporal dependencies and patterns over time.

Two examples of sequence-based tasks are:

1. **Machine Translation:** This is a classic sequence-to-sequence task. The input is a sequence of words in a source language (e.g., "How are you?"), and the model's output is a corresponding sequence of words in a target language (e.g., "Comment allez-vous ?"). The order of words is fundamental to the sentence's meaning.
2. **Sentiment Analysis:** This is often a sequence-to-one task. The input is a sequence of words, such as a movie review ("This movie was a fantastic and compelling story."), and the output is a single label that classifies the sentiment of the entire sequence (e.g., "Positive"). The model must understand the context built up by the sequence of words to make its final classification.

---

2. Differentiate between sequence-to-sequence and sequence-to-one models.

These models are differentiated by the nature of their output.

Feature	Sequence-to-One (Many-to-One)	Sequence-to-Sequence (Many-to-Many)
Input Type	A sequence of data points (e.g., words, time-series data).	A sequence of data points.
Output Type	A single, fixed-size output (e.g., a class label, a score).	A sequence of data points, which can be of a different length than the input.
Core Task	To **classify** or **summarize** an entire input sequence.	To **transform** an input sequence into an output sequence.
Typical Architecture	An RNN reads the entire input sequence and uses its final hidden state to make a prediction through a feedforward layer.	Typically uses an **Encoder-Decoder** architecture. The encoder processes the entire input sequence into a context vector, and the decoder generates the output sequence from that context.
Examples	- **Sentiment Analysis** (sequence of	- **Machine Translation** (sentence ->

	words -> positive/negative) - **Video Classification** (sequence of frames -> action label)	translated sentence) - **Speech Recognition** (audio sequence -> text sequence)

---

3. Why are RNNs suitable for sequence learning compared to feedforward networks?

RNNs are fundamentally better suited for sequence learning than feedforward networks (FNNs) due to their unique architectural features that address the limitations of FNNs.

1. **Internal Memory (Hidden State):** The most crucial feature of an RNN is its **hidden state**, which acts as a form of memory. At each time step, the RNN's output is a function of both the current input and the hidden state from the previous time step. This allows the network to retain information from past elements in the sequence and use it to process future elements. FNNs, in contrast, have no memory; they treat every input as an independent event.
2. **Parameter Sharing Across Time:** An RNN uses the same set of weights and biases for every time step in the sequence. This has two major benefits:
    - It allows the model to handle **variable-length sequences** gracefully, as the same mechanism is applied regardless of the sequence's length.
    - It enables the model to learn patterns that can occur at any point in the sequence. A pattern learned at the beginning of a sequence can be recognized if it appears at the end. FNNs would require separate weights for each position, which is inefficient and not scalable.
3. **Handling of Temporal Dependencies:** The recurrent loop structure inherently models the temporal flow of data, making RNNs naturally equipped to capture dependencies and relationships between elements that are far apart in the sequence. FNNs require a fixed-size input and lose all sequential ordering when data is flattened into a vector.

---

4. What is a computational graph? How is it used in training RNNs?

A **computational graph** is a directed graph used to represent a mathematical computation. In the context of neural networks, nodes in the graph represent variables (like inputs, weights) or operations (like matrix multiplication, addition, activation functions), and the edges represent the flow of data between these nodes.

How it's used in training RNNs:

The recurrent nature of an RNN, with its feedback loop, makes it difficult to apply standard backpropagation directly. To solve this, the RNN is "unrolled" (or "unfolded") in time to create a computational graph.

1. **Unrolling the Graph:** The compact, cyclic graph of an RNN is transformed into a deep, acyclic feedforward network structure. Each time step of the sequence becomes a distinct "layer" in this unrolled graph. The hidden state from time step $t-1$ is passed as an input to the layer for time step t. All these time-step layers share the same set of weights.
2. **Backpropagation Through Time (BPTT):** Once the RNN is unrolled, it behaves like a very deep feedforward network. This allows the standard backpropagation algorithm to be used for training. This specific application is called **Backpropagation Through Time (BPTT)**.
    - A forward pass is performed through the unrolled graph to compute the output and the loss at

each time step.
    - A backward pass is then performed. Gradients of the total loss are calculated and propagated backward from the last time step to the first.
    - Because the weights are shared across all time steps, the gradients for each weight matrix are summed up across all time steps before the weights are updated. This single update step incorporates the influence of the weight on the output across the entire sequence.

---

5. Differentiate between static and dynamic computational graphs.

Feature	Static Computational Graph	Dynamic Computational Graph
Definition	The graph structure is defined and compiled **once** before the model is run. The same graph is then executed repeatedly with different input data.	The graph structure is built **on-the-fly** as the code executes. The graph can be different for every iteration or input sample.
Flexibility	**Rigid.** The architecture is fixed. Handling variable-length inputs (like in RNNs) requires special techniques like padding and bucketing.	**Highly Flexible.** The graph can change dynamically based on the control flow of the programming language (e.g., loops, if-statements). This makes it very natural to handle variable-length sequences.
Debugging	**More difficult.** Errors often occur during graph compilation or execution, far from the code that defined the graph. Debugging requires specialized tools.	**Easier.** Since the graph is built during execution, you can use standard debuggers to set breakpoints and inspect values at any point in the forward pass.
Performance	Can be **highly optimized** by the framework before execution, as the entire graph is known. This often leads to better performance and memory efficiency.	May have slightly more overhead due to the repeated graph construction, but modern frameworks have largely minimized this gap.
Framework Examples	TensorFlow 1.x, Theano	PyTorch, TensorFlow 2.x (in Eager Mode)

---

6. Explain the architecture of a bidirectional RNN.

A **Bidirectional RNN (BiRNN)** is an architecture designed to capture context from both past and future elements in a sequence for any given point in time. A standard (unidirectional) RNN only considers past context, which can be a limitation for many tasks.

Architecture:
A BiRNN consists of two independent RNN layers that process the input sequence in opposite directions.

1. **Forward Layer:** This is a standard RNN layer that reads the input sequence from left to right (from

time step t=1 to t=T). It produces a sequence of forward hidden states: (overrightarrowh_1,overrightarrowh_2,...,overrightarrowh_T). Each state overrightarrowh_t summarizes the past information up to time t.

2. **Backward Layer:** This is another RNN layer that reads the input sequence from right to left (from time step t=T to t=1). It produces a sequence of backward hidden states: (overleftarrowh_1,overleftarrowh_2,...,overleftarrowh_T). Each state overleftarrowh_t summarizes the future information from time t onwards.
3. Output: At each time step t, the final hidden state representation, h_t, is created by combining the outputs from both layers. This combination is typically done by concatenation, but summation or averaging can also be used.

   `ht=[ht;ht]`

This combined representation h_t provides a rich summary of the input at position t, incorporating information from its entire context.

---

7. Mention one advantage and one limitation of bidirectional RNNs.

- Advantage: Complete Contextual Understanding
  The primary advantage of a BiRNN is its ability to access both past (preceding) and future (succeeding) information for every point in the sequence. This provides a much richer and more complete contextual representation, leading to significantly better performance on tasks like Named Entity Recognition (NER), machine translation, and text summarization, where the meaning of a word is often clarified by the words that follow it.
- Limitation: Unsuitable for Real-Time Prediction
  The main limitation is that a BiRNN is not causal; it requires the entire input sequence to be available before it can make a prediction for any time step. This is because the backward layer must process the sequence from the very end. Therefore, BiRNNs cannot be used for real-time applications where predictions must be made online as data arrives, such as live speech transcription or stock price prediction.

---

9. What is the vanishing gradient problem in RNNs?

The **vanishing gradient problem** is a critical issue that occurs when training deep neural networks, and it is particularly severe in RNNs processing long sequences. It describes a situation where the gradients of the loss function with respect to the network's early-layer weights become exponentially small (i.e., they "vanish").

Mechanism in RNNs:
During Backpropagation Through Time (BPTT), the gradient is propagated backward from the end of the sequence to the beginning. To do this, it is repeatedly multiplied by the recurrent weight matrix (W_hh) at each time step.
If the values in this matrix are small (more formally, if its leading eigenvalue is less than 1), these repeated multiplications cause the gradient signal to shrink exponentially as it travels back through time.

Consequence:
When the gradient becomes vanishingly small, the weight updates for the earlier time steps are

negligible. This means the model cannot learn long-range dependencies—it struggles to connect events that are far apart in the sequence. For example, in the sentence "The writer of the books... is famous," the model would struggle to learn the grammatical agreement between "writer" and "is" because the gradient from "is" would vanish before it could reach and update the weights associated with processing "writer."

---

10. Explain exploding gradients with an example.

Exploding gradients is the opposite problem to vanishing gradients. It occurs when the gradients grow exponentially large, leading to massive, unstable weight updates during training. This can cause the model's weights to become so large that they are represented as NaN (Not a Number), effectively destroying the model and halting the training process.

Mechanism in RNNs:
Similar to the vanishing gradient problem, this occurs during BPTT due to the repeated multiplication of the gradient by the recurrent weight matrix (W_hh). If the values in this matrix are large (leading eigenvalue > 1), the gradient signal will grow exponentially as it is propagated backward through time.
Example:
Consider a very simple RNN with a single recurrent neuron where the update rule is $h_t = w_{hh} \cdot h_{t-1}$. When we compute the gradient of the loss at time t with respect to a much earlier hidden state h_k (where $k \ll t$), the chain rule dictates that the gradient will contain the term:

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} = \prod_{i=k+1}^{t} w_{hh} = (w_{hh})^{t-k}$$

If the absolute value of the weight $|w_{hh}|$ is greater than 1 (e.g., $w_{hh}=1.5$), this term will grow exponentially as the distance $(t-k)$ increases. A small error at the end of the sequence will be amplified into a massive gradient for the beginning of the sequence, causing the weights to "explode."
The most common technique to combat this is **gradient clipping**, where the gradients are manually scaled down if their norm exceeds a certain predefined threshold.

---

11. Draw and explain the basic architecture of an LSTM cell.

An **LSTM (Long Short-Term Memory)** cell is a sophisticated type of recurrent unit designed specifically to overcome the vanishing gradient problem and learn long-range dependencies. It achieves this using a system of gates to regulate the flow of information.

Core Components and Explanation:

The key to an LSTM is the **cell state (C_t)**, which acts as a memory "conveyor belt." Information can flow along it largely unchanged, making it easy for gradients to pass through many time steps. The flow is controlled by three main gates:

1. Forget Gate (f_t): This gate decides what information to discard from the previous cell state (C_{t-1}). It looks at the previous hidden state (h_{t-1}) and the current input (x_t) and outputs a number between 0 and 1 for each number in the cell state. A 1 means "completely keep this," while a 0 means "completely forget this."

`ft=σ(Wf·[ht-1,xt]+bf)`

2. **Input Gate (i_t):** This gate decides what new information to store in the cell state. It has two parts:
   - A sigmoid layer (i_t) decides which values to update.
   - A tanh layer creates a vector of new candidate values (tildeC∗t) that could be added.

   `it=σ(Wi·[ht-1,xt]+bi)`
   `C~t=tanh(WC·[ht-1,xt]+bC)`

   The old cell state is then updated by forgetting some information ($f\_t * C*{t-1}$) and adding the new candidate information ($i\_t * \\tilde{C}\_t$).

   `Ct=ft⊙Ct-1+it⊙C~t`
3. Output Gate (o_t): This gate decides what part of the cell state will be output as the new hidden state (h_t). The cell state is passed through a tanh function (to squash values between -1 and 1) and then multiplied by the output of the sigmoid gate (o_t).

   `ot=σ(Wo·[ht-1,xt]+bo)`
   `ht=ot⊙tanh(Ct)`

This gating mechanism allows LSTMs to selectively remember or forget information over long periods, making them highly effective for complex sequence tasks.

---

13. Differentiate between LSTM and vanilla RNN.

Feature	Vanilla RNN (Simple RNN)	LSTM (Long Short-Term Memory)
Internal Structure	Very simple, consisting of a single recurrently connected layer, typically with a tanh or ReLU activation function.	Complex, containing an internal **cell state** and three regulatory **gates** (forget, input, and output).
Memory Handling	Has a single hidden state that serves as its short-term memory. This state is completely overwritten at each time step.	Maintains both a **hidden state** (short-term memory) and a **cell state** (long-term memory). The gates control what is added to or removed from this long-term memory.
Gradient Flow	Highly susceptible to the **vanishing and exploding gradient problems** due to repeated matrix multiplications in the backpropagation path.	The additive nature of the cell state update provides a clear path for gradients to flow, which significantly **mitigates the vanishing gradient problem**.
Capability	Effective at learning short-term dependencies but fails on tasks requiring the model to remember information over long sequences.	Specifically designed to learn **long-range dependencies**, making it highly effective for complex tasks like machine translation and speech recognition.

Computational Cost	Computationally cheaper and has fewer parameters.	More complex and computationally expensive due to the additional gating mechanisms and parameters.

---

14. How can RNNs be applied in self-driving cars?

Self-driving cars operate in a dynamic environment where understanding sequences of events over time is crucial for safety and decision-making. RNNs (and their variants like LSTMs and GRUs) are applied to model these temporal aspects.

1. **Trajectory Prediction:** One of the most critical tasks is predicting the future path of other agents (vehicles, pedestrians, cyclists). An RNN can take a sequence of past positions, velocities, and accelerations of an object as input and output a sequence of its most likely future coordinates. This is essential for anticipatory driving and collision avoidance.
2. **Sensor Fusion over Time:** A self-driving car is equipped with multiple sensors (cameras, LiDAR, radar). An RNN can be used to fuse the data streams from these sensors over time. By processing sequences of sensor readings, the model can build a more stable, robust, and noise-resistant representation of the surrounding environment, improving object detection and tracking.
3. **Behavior Prediction:** RNNs can be used to classify and predict the intent of other drivers or pedestrians. For example, by analyzing a sequence of a vehicle's movements (slight drift, turn signal status, speed changes), an RNN can predict the probability of an imminent lane change or turn.

---

15. Mention two sequence-learning problems faced by self-driving cars.

Self-driving cars must solve numerous complex sequence-learning problems to navigate safely. Two key examples are:

1. **Pedestrian Intent Prediction:** This is the problem of predicting a pedestrian's next action based on their recent behavior.
    - **Input:** A sequence of observations, such as video frames capturing their body posture, walking direction, speed, and head orientation over the last few seconds.
    - **Output:** A classification of their intent (e.g., 'will cross the street', 'will wait', 'is unaware of the car').
    - **Challenge:** Pedestrian behavior can be highly unpredictable and change abruptly. The model must learn subtle cues from the sequence to make a safe and timely decision. This is a critical sequence-to-one problem.
2. **Vehicle Trajectory Forecasting in Complex Scenarios:** This is the problem of predicting the future path of other vehicles, especially in dense traffic or at intersections.
    - **Input:** A sequence of the vehicle's past states (position, velocity, acceleration) and contextual information (e.g., traffic light status, road geometry).
    - **Output:** A sequence of future coordinates representing the vehicle's likely path over the next 3-5 seconds.
    - **Challenge:** A vehicle's future trajectory is influenced not just by its own history but also by the interactive behavior of all surrounding vehicles. The model must learn these complex multi-agent dynamics. This is a challenging sequence-to-sequence problem.

---

8. What is Backpropagation Through Time (BPTT)? Why is it needed for RNNs?

Backpropagation Through Time (BPTT) is the standard algorithm used to train Recurrent Neural Networks. It is an adaptation of the standard backpropagation algorithm applied to the "unrolled" version of an RNN.

Why is it needed?
A standard backpropagation algorithm works on feedforward networks, which have a clear, acyclic path from input to output. RNNs, however, contain cycles (or feedback loops) where the output of a time step is fed back as an input to the next. Standard backpropagation cannot handle these loops. BPTT solves this by creating a feedforward representation of the RNN, allowing gradients to be calculated.
The Process:

1. **Unroll the Network:** The first step is to unroll the RNN's recurrent structure through time. This transforms the network with a cycle into a very deep feedforward network, where each time step of the input sequence corresponds to one layer in the unrolled network. Critically, the **weights are shared** across all these time-step layers.
2. **Forward Pass:** An input sequence is passed through the unrolled network, and the outputs are calculated at each time step. The error (loss) is then computed by comparing the network's predictions to the true target sequence.
3. **Backward Pass:** The gradient of the loss is then calculated and propagated backward from the last time step to the first, just like in a standard deep feedforward network.
4. **Weight Update:** Because the weights (Wxh, Whh, Why) are shared across all time steps, the gradients for each shared weight are summed up across all the unrolled steps. Finally, the weights are updated using this aggregated gradient.

A limitation of BPTT is that for very long sequences, the unrolled graph becomes extremely deep, which is computationally expensive and can lead to vanishing or exploding gradients. To manage this, a truncated version (Truncated BPTT) is often used, where backpropagation is only run for a fixed number of recent time steps.

---

12. Explain the architecture of a Gated Recurrent Unit (GRU).

A **Gated Recurrent Unit (GRU)** is a type of recurrent neural network cell, introduced as a simpler and more computationally efficient alternative to the LSTM. Like LSTMs, GRUs use gating mechanisms to control the flow of information and effectively combat the vanishing gradient problem, allowing them to capture long-range dependencies.

Architecture and Key Differences from LSTM:
A GRU simplifies the LSTM architecture in two main ways:

- It merges the cell state and the hidden state into a **single hidden state vector (ht)**.
- It combines the LSTM's forget and input gates into a single **"update gate"**.

A GRU cell consists of two primary gates:

1. **Update Gate (zt):** This gate determines how much of the past information (from the previous hidden state ht−1) should be carried forward to the future. It acts like a combination of the forget

and input gates from an LSTM.

- It calculates a value between 0 and 1. A value close to 1 means the previous hidden state is almost entirely passed through, while a value close to 0 means the new candidate state is used instead.

  $$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

2. **Reset Gate (rt):** This gate determines how much of the past information to *forget* when creating the new candidate hidden state. This allows the cell to drop information that is found to be irrelevant for the future.
   - It also calculates a value between 0 and 1. If the value is close to 0, it effectively makes the cell act as if it's reading the first input of a new sequence, allowing it to forget the previously computed state.

     $$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

These gates are then used to compute the new hidden state ht. The reset gate decides how to combine the previous state with the current input to create a candidate state (h~t), and the update gate decides how to mix that candidate state with the original previous state (ht−1) to get the final output.

Benefit: With fewer parameters and operations than an LSTM, a GRU is computationally faster and may perform better on smaller datasets where it is less likely to overfit.